DACC | Departamento Acadêmico de Ciência da Computação

FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA

Programação Orientada a Objetos

Professor:
Dr. Lucas Marques da Cunha
lucas.marques@unir.br

UNIR

Guia Visual de Sobrevivência e Introdução ao Java

Fundamentos, Ecossistema e Programação Orientada a Objetos

DACC Departamento Acadêmico de Ciência da Computação
FUNDAÇÃO UNIVERSIDADE FEDERAL DE RONDÔNIA
UNIR

NotebookLM

A Essência e o DNA do Java

Top Linguagens para 2025

Visual Timeline

1991: O Nascimento

Criada na Sun Microsystems através do Green Project, liderado por James Gosling.

Orientada a Objetos

Modelagem de software baseada em objetos e classes, facilitando reutilização e manutenção.

Portável e Independente

Write Once, Run Anywhere (WORA). O bytecode Java roda na Java Virtual Machine (JVM) em qualquer plataforma.

Robusta e Segura

Foco em confiabilidade com gerenciamento de memória automático, tratamento de exceções e mecanismos de segurança embutidos.

Fortemente Tipada

Verificação de tipos rigorosa em tempo de compilação, reduzindo erros e melhorando a estabilidade do código.

NotebookLM

O Ecossistema Descomplicado

1. JDK (Java Development Kit)

O kit do construtor. Contém o **compilador** e as **bibliotecas** para **criar o software.**

2. JRE (Java Runtime Environment)

O palco de execução. Contém a JVM e as APIs necessárias para rodar aplicações prontas.

3. IDE (Integrated Development Environment)

A oficina do programador. Ferramentas visuais.

NetBeans | eclipse | IntelliJ IDEA

NotebookLM

As Três Fronteiras da Plataforma

Java EE
(Enterprise Edition)

O mundo corporativo. Focado em servidores de aplicação e sistemas web complexos.

Java SE
(Standard Edition)

O core do ecossistema. Base da linguagem, ideal para aplicações desktop, redes e processamento paralelo.

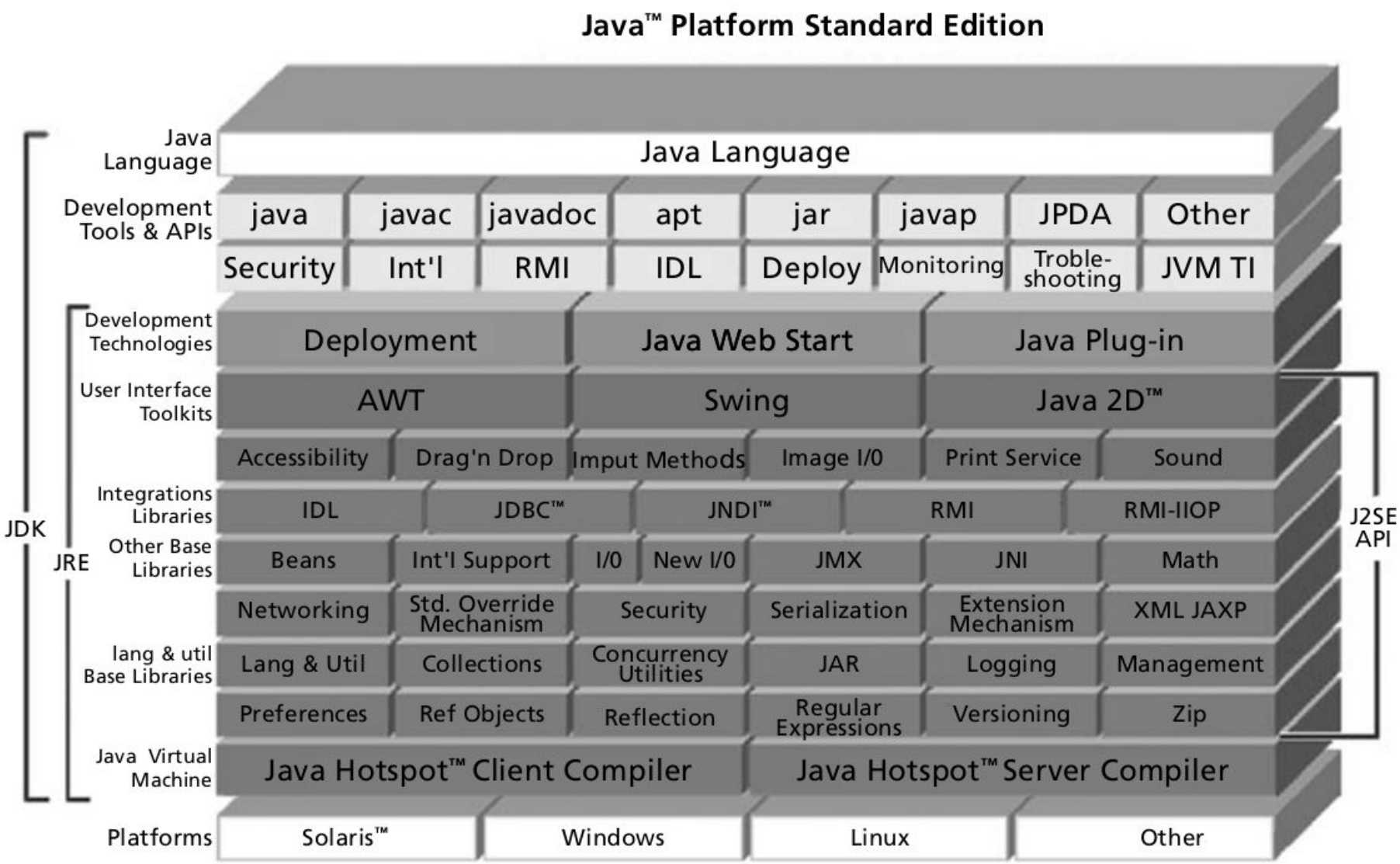
Java SE Development

Java ME
(Micro Edition)

O mundo embarcado. Otimizado para dispositivos móveis e sistemas restritos.

NotebookLM

Linguagens Orientadas a Objetos



A Mágica da Portabilidade: Compilação e JVM

1. Código Fonte
Arquivo .java (Legível por humanos)

2. Compilador
javac transforma o texto

3. Bytecode
Arquivo .class (O "Assembly" do Java)

4. JVM
A Java Virtual Machine interpreta o bytecode

Execução
Traduzido para o sistema operacional local

NotebookLM

O Operário Invisível: Garbage Collection

Responsável pela liberação automática do espaço em memória durante a vida do programa.

- Opera silenciosamente em segundo plano.
- Examina periodicamente a memória dinâmica (heap).
- Identifica e "limpa" objetos sem referências ativas.

NotebookLM

Dissecando o Primeiro Programa

```
public class Universe {
    public static void main(String[] args) {
        System.out.println("Hello Universe!");
    }
}
```

Todo mundo pode executar.

Todo código Java deve pertencer a uma classe (nome da classe).

Não retorna nada.

O nome do método e seus parâmetros (arranjo de strings).

Comando de impressão seguido de ponto e vírgula.

Chaves delimitam corpos de classes e métodos.

Método pertence à classe, não ao objeto.

NotebookLM

Compilação e Execução na Prática

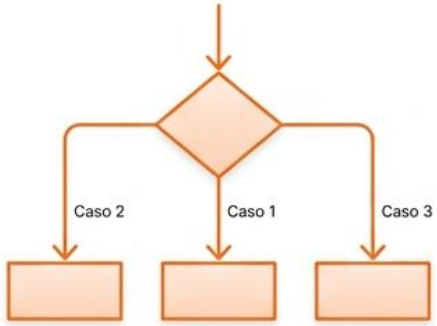
Terminal

```
$ javac PrimeiraClasse.java
(Gera o Bytecode em background)

$ java PrimeiraClasse
Esse é o nosso primeiro programa...
```

Atenção: Nunca incluir a extensão .class ou .java na hora de executar com o comando java!

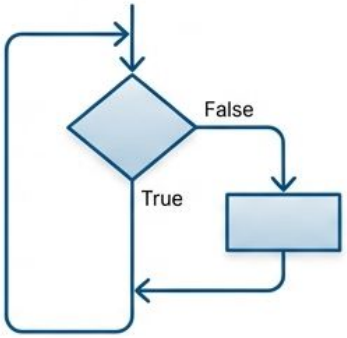
NotebookLM

Estruturas de Controle de Fluxo



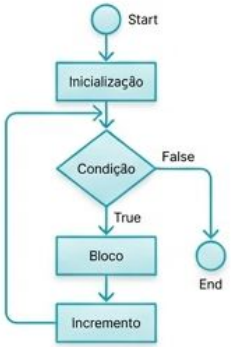
Seleção (switch)

Avalia múltiplos casos. Suporta tipos numéricos, char, String e enum.



Repetição Contínua (while)

Executa o bloco enquanto uma condição for verdadeira. Ideal para laços indefinidos.



Repetição Contada (for)

Sintaxe condensada (inicialização; condição; incremento). Ideal para iterações com limites claros.

NotebookLM

Modularização Através de Métodos

Em Java, "funções" são chamadas de métodos. São blocos de código projetados para realizar uma tarefa específica.

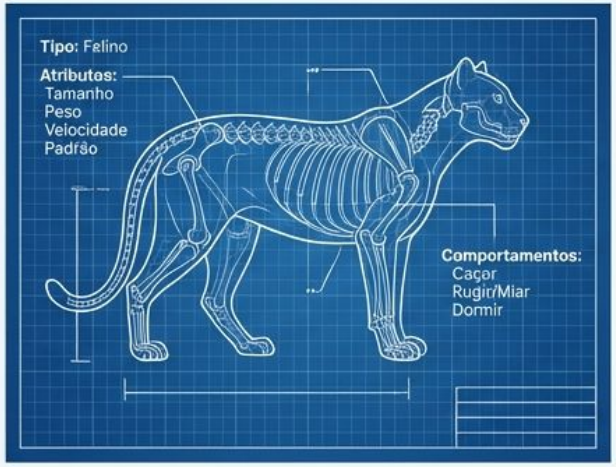
- Encapsulam a lógica de processamento.
- Promovem a reutilização (chame o método, não reescreva).
- Facilitam a manutenção em sistemas complexos.

NotebookLM

O Salto para a Orientação a Objetos

A Analogia Biológica: A palavra “classe” vem da taxonomia da biologia.

A Classe (O Molde)



Define os atributos e comportamentos comuns que uma família possui. (Ex: “Classe dos Felinos”).

O Objeto (O Indivíduo)

Ex: “Leão” (Instância 1) -
Tamanho: Grande
Padrão: Juba
Comportamento: Ruge

Ex: “Tigre” (Instância 2) -
Tamanho: Grande
Padrão: Listrado
Comportamento: Ruge/Miar

Uma instância da classe. Possui as mesmas características base, mas com valores únicos. (Ex: “Leão”, “Tigre”).

NotebookLM

Da Teoria à Prática: Desenhando a Classe Produto

Classe: Produto

Atributos (O que ele tem):

- quantidade
- descrição
- preço

Métodos (O que ele faz):

- verQuantidade()
- verDescrição()
- verPreço()
- calcularValorEstoque()

Objeto 1(Teclado)

Instância da classe Produto.
Com valores específicos.

Objeto 2 (Monitor)

Instância da classe Produto.
Com valores específicos.

Objeto 3 (Mouse)

Instância da classe Produto.
Com valores específicos.

Uma única classe arquitetural cria **múltiplas instâncias de objetos exclusivos**.

NotebookLM

Desafios Práticos e Próximos Passos

Missão 1: O Scanner

Ler três inteiros usando Scanner, calcular o produto, e usar System.out.printf para exibir o resultado.

Missão 2: Lógica Aplicada (IMC)

Receber peso e altura, calcular o Índice de Massa Corporal e tabelar a saúde do usuário.

Missão 3: O Laço Matemático

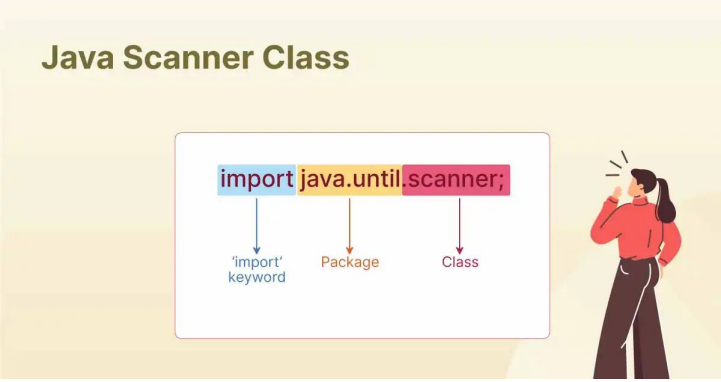
Programar a Conjectura de Collatz (Se par: x/2; Se ímpar: 3x+1) com um laço while até chegar a 1.

NotebookLM

VAMOS
PROGRAMAR?

JAVA: Entrada padrão

- Como atribuir valores para uma variável usando o teclado? Em C, por exemplo, tem-se a função scanf() e em Pascal, o procedimento readln().
- No Java está disponível a classe **Scanner** do pacote java.util. Essa classe implementa as operações de entrada de dados pelo teclado no console.



JAVA: Entrada padrão

```java
// Figura 2.7: Addition.java
// Programa de adição que insere dois números, então exibe a soma deles.
import java.util.Scanner; // programa utiliza a classe Scanner

public class Addition
{
   // método main inicia a execução do aplicativo Java
   public static void main(String[] args)
   {
      // cria um Scanner para obter entrada a partir da janela de comando
      Scanner input = new Scanner(System.in);

      int number1; // primeiro número a somar
      int number2; // segundo número a somar
      int sum; // soma de number1 e number2

      System.out.print("Enter first integer: "); // prompt
      number1 = input.nextInt(); // lê primeiro o número fornecido pelo usuário

      System.out.print("Enter second integer: "); // prompt
      number2 = input.nextInt(); // lê o segundo número fornecido pelo usuário

      sum = number1 + number2; // soma os números, depois armazena o total em sum

      System.out.printf("Sum is %d%n", sum); // exibe a soma
   } // fim do método main
} // fim da classe Addition
```

JAVA: Estrutura de Seleção

- No Java, a sintaxe do if é a seguinte:

```
if (condicaoBooleana) {
    codigo;
}
```

- Além disso, você pode usar a cláusula else para indicar o comportamento que deve ser executado no caso da expressão booleana ser falsa:

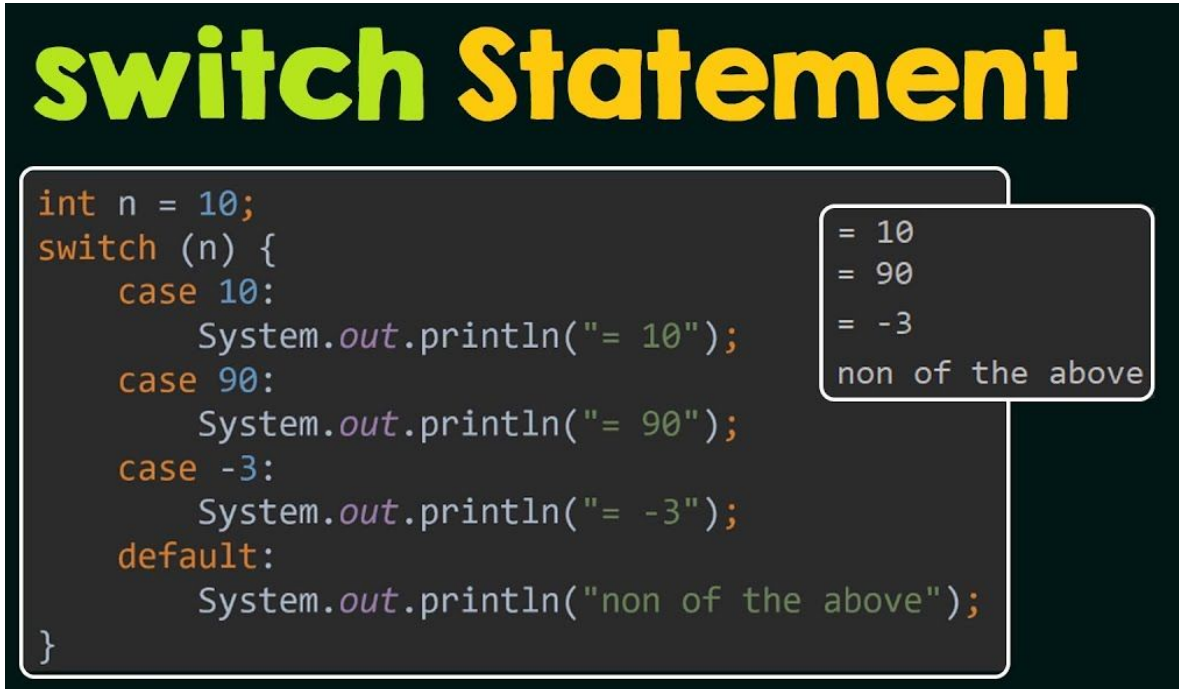
```
int idade = 15;
if (idade < 18) {
    System.out.println("Não pode entrar");
} else {
    System.out.println("Pode entrar");
}
```

JAVA: Estrutura de Seleção

```java
// Figura 2.15: Comparison.java
// Compara inteiros utilizando instruções if, operadores relacionais
// e operadores de igualdade.
import java.util.Scanner; // programa utiliza a classe Scanner

public class Comparison
{
   // método main inicia a execução do aplicativo Java
   public static void main(String[] args)
   {
      // cria Scanner para obter entrada a partir da linha de comando
      Scanner input = new Scanner(System.in);

      int number1; // primeiro número a comparar
      int number2; // segundo número a comparar

      System.out.print("Enter first integer: "); // prompt
      number1 = input.nextInt(); // lê o primeiro número fornecido pelo usuário

      System.out.print("Enter second integer: "); // prompt
      number2 = input.nextInt(); // lê o segundo número fornecido pelo usuário

      if (number1 == number2)
         System.out.printf("%d == %d%n", number1, number2);

      if (number1 != number2)
         System.out.printf("%d != %d%n", number1, number2);

      if (number1 < number2)
         System.out.printf("%d < %d%n", number1, number2);

      if (number1 > number2)
         System.out.printf("%d > %d%n", number1, number2);

      if (number1 <= number2)
         System.out.printf("%d <= %d%n", number1, number2);

      if (number1 >= number2)
         System.out.printf("%d >= %d%n", number1, number2);
   } // fim do método main
} // fim da classe Comparison
```

JAVA: Estrutura de Seleção

- É importante notar que a expressão usada no switch pode ser de tipo byte, short, int, char, String ou um enum. A partir do Java 7, também é possível usar switch com tipos long, desde que cada case tenha um literal constante inteiro que caiba dentro de um tipo int.



JAVA: Estrutura de Seleção

```
1 import java.util.Scanner;
2
3 public class ExemploSwitchString {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6         System.out.print("Digite o nome de um mês: ");
7         String mes = scanner.nextLine().toLowerCase();
8         String estacao;
9         switch(mes) {
10            case "janeiro":
11            case "fevereiro":
12            case "março":
13                estacao = "Verão";
14                break;
15            case "abril":
16            case "maio":
17            case "junho":
18                estacao = "Outono";
19                break;
20            case "julho":
21            case "agosto":
22            case "setembro":
23                estacao = "Inverno";
24                break;
25            case "outubro":
26            case "novembro":
27            case "dezembro":
28                estacao = "Primavera";
29                break;
30            default:
31                estacao = "Mês inválido";
32        }
33        System.out.println("A estação correspondente ao mês de " + mes + " é " + estacao + ".");
34        scanner.close();
35    }
36 }
37
```

JAVA: Estrutura de repetição

- No Java, a sintaxe do **while** é a seguinte:

```
int idade = 15;
while (idade < 18) {
    System.out.println(idade);
    idade = idade + 1;
}
```

- No Java, a sintaxe do **for** é a seguinte:

```
for (int i = 0; i < 10; i = i + 1) {
    System.out.println("olá!");
}
```

JAVA: Estrutura de repetição

```
1 public class TabuleiroDamas {
2     public static void main(String[] args) {
3         int tamanho = 8;
4         for (int i = 0; i < tamanho; i++) {
5             for (int j = 0; j < tamanho; j++) {
6                 if ((i + j) % 2 == 0) {
7                     System.out.print("■ ");
8                 } else {
9                     System.out.print("□ ");
10                }
11            }
12            System.out.println();
13        }
14    }
15 }
```

JAVA: Estrutura de repetição

```
1 import java.util.Scanner;
2 import java.util.Random;
3
4 public class ExemploWhile {
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7         Random random = new Random();
8         int numeroAleatorio = random.nextInt(100) + 1; // Gera um número aleatório entre 1 e 100
9         System.out.println("Tente adivinhar o número entre 1 e 100!");
10        int tentativa;
11        int tentativas = 0;
12        while (true) {
13            System.out.print("Digite seu palpite: ");
14            tentativa = scanner.nextInt();
15            tentativas++;
16            if (tentativa == numeroAleatorio) {
17                System.out.println("Parabéns! Você acertou o número em " + tentativas + " tentativas.");
18                break; // Sai do loop
19            } else if (tentativa < numeroAleatorio) {
20                System.out.println("Tente um número maior.");
21            } else {
22                System.out.println("Tente um número menor.");
23            }
24        }
25
26        scanner.close();
27    }
28 }
```

JAVA: Estruturas homogêneas

- **Vetores** são estruturas de dados que armazenam usualmente uma quantidade fixa de dados de um certo tipo; por esta razão, também são conhecidos como estruturas homogêneas de dados.
- Na declaração de vetores deverão ser fornecidas três informações: 1) o nome do vetor, 2) o número de posições do vetor (seu tamanho), e 3) o tipo de dado que será armazenado no vetor. A declaração de um vetor para "inteiros", de nome "vetor", em Java:

```
1 | int vetor[] = new int[10]; // declaração combinada
```

JAVA: Estruturas homogêneas

- **Exemplo 01:**

```java
int n = 10; // tamanho do vetor
int v[] = new int[n]; // declaração e alocação de espaço para o vetor "v"
int i; // índice ou posição

// processando os "n" elementos do vetor "v"
for (i=0; i<n; i++) {
  v[i] = i; // na i-ésima posição do vetor "v" armazena o valor da variável "i"
}
```

JAVA: Estruturas homogêneas

- **Exemplo 02 - Estrutura for aprimorada:**

```java
class ExemploVetor{

    public static void main (String [] args){
        String semaforo[] = {"vermelho", "amarelo", "verde"};
        System.out.println("Ordem de um Semáforo: \n");
        for (String sinal: semaforo){
            System.out.printf("%s\n", sinal);
        }
    }

}
```

JAVA: Função

- Em Java, uma "função" é um conceito representado por métodos.
- **Métodos** são blocos de código que realizam uma tarefa específica e podem ser chamados (ou invocados) a partir de outras partes do programa.
- Eles são usados para encapsular a lógica de processamento e modularizar o código, o que promove a reutilização e a manutenção.

JAVA: Função

```java
1 import java.util.Scanner;
2
3 public class ExemploFuncao {
4     // Função para calcular a área de um círculo
5     public static double calcularArea(double raio) {
6         return Math.PI * raio * raio;
7     }
8     // Função para calcular o perímetro de um círculo
9     public static double calcularPerimetro(double raio) {
10        return 2 * Math.PI * raio;
11    }
12    public static void main(String[] args) {
13        Scanner scanner = new Scanner(System.in);
14        System.out.print("Digite o raio do círculo: ");
15        double raio = scanner.nextDouble();
16        // Chamando a função para calcular a área
17        double area = calcularArea(raio);
18        // Chamando a função para calcular o perímetro
19        double perimetro = calcularPerimetro(raio);
20        System.out.println("Área do círculo: " + area);
21        System.out.println("Perímetro do círculo: " + perimetro);
22        scanner.close();
23    }
24 }
```

Atividade prática

Atividade prática

1. **Escreva declarações, instruções ou comentários que realizem cada uma das tarefas a seguir:**

   a) Declare que um programa calculará o produto de três inteiros.

   b) Crie um Scanner chamado input que leia valores a partir da entrada padrão.

   c) Declare as variáveis x, y, z e result como tipo int.

   d) Solicite que o usuário insira o primeiro inteiro.

   e) Leia o primeiro inteiro digitado pelo usuário e armazene-o na variável x.

   f) Solicite que o usuário insira o segundo inteiro.

   g) Leia o segundo inteiro digitado pelo usuário e armazene-o na variável y.

   h) Solicite que o usuário insira o terceiro inteiro.

   i) Leia o terceiro inteiro digitado pelo usuário e armazene-o na variável z.

   j) Compute o produto dos três inteiros contidos nas variáveis x, y e z e atribua o resultado à variável result.

   k) Use System.out.printf para exibir a mensagem “Product is” seguida pelo valor da variável result.

Atividade prática

2. **Escreva um aplicativo que calcule os quadrados e cubos dos números de 0 a 10 e imprima os valores resultantes em formato de tabela como a seguir:**

```
number    square    cube
0         0         0
1         1         1
2         4         8
3         9         27
4         16        64
5         25        125
6         36        216
7         49        343
8         64        512
9         81        729
10        100       1000
```

Atividade prática

3. **Crie um programa de calculadora IMC que leia o peso do usuário em libras e a altura em polegadas (ou, se preferir, o peso em quilogramas e a altura em metros) e, então, calcule e exiba o índice de massa corporal dele. Além disso, que exiba as seguintes informações do Department of Health and Human Services/National Institutes of Health, assim o usuário pode avaliar o seu IMC:**

```
BMI VALUES
Underweight: less than 18.5
Normal:      between 18.5 and 24.9
Overweight:  between 25 and 29.9
Obese:       30 or greater
```

$$\text{IMC} = \frac{pesoEmLibras \times 703}{alturaEmPolegadas^2}$$

ou

$$\text{IMC} = \frac{pesoEmQuilogramas}{alturaEmMetros^2}$$

Atividade prática

4. **Escreva um programa em que, dada uma variável x com algum valor inteiro, temos um novo x de acordo com a seguinte regra:**
   - Se x é par, x = x / 2 ;
   - Se x é impar, x = 3 * x + 1 ;
   - Imprime x ;

   O programa deve parar quando x tiver o valor final de 1. Por exemplo, para x = 13 , a saída será:

   40 -> 20 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1

Atividade prática

5. **Escreva um programa que leia a entrada a partir do usuário do raio de um círculo como um inteiro e imprima o diâmetro do círculo, circunferência e área utilizando o valor do ponto flutuante 3,14159 para S. [Observação: você também pode empregar a constante Math.PI predefinida para o valor de S.**

   Adote as seguintes fórmulas (r é o raio) para implementar os métodos:

$$diâmetro = 2r$$
$$circunferência = 2\pi r$$
$$área = \pi r^2$$

ORIENTAÇÃO A OBJETOS BÁSICA

- Considere um programa para uma escola. É bem fácil perceber que uma entidade extremamente importante ao nosso sistema é o aluno. Nossa ideia aqui é generalizarmos alguma informação juntamente com as funcionalidades que todo aluno deve ter.

- O que todo aluno tem que é importante para nós?
  - Número da matrícula;
  - Nome do aluno;
  - curso.

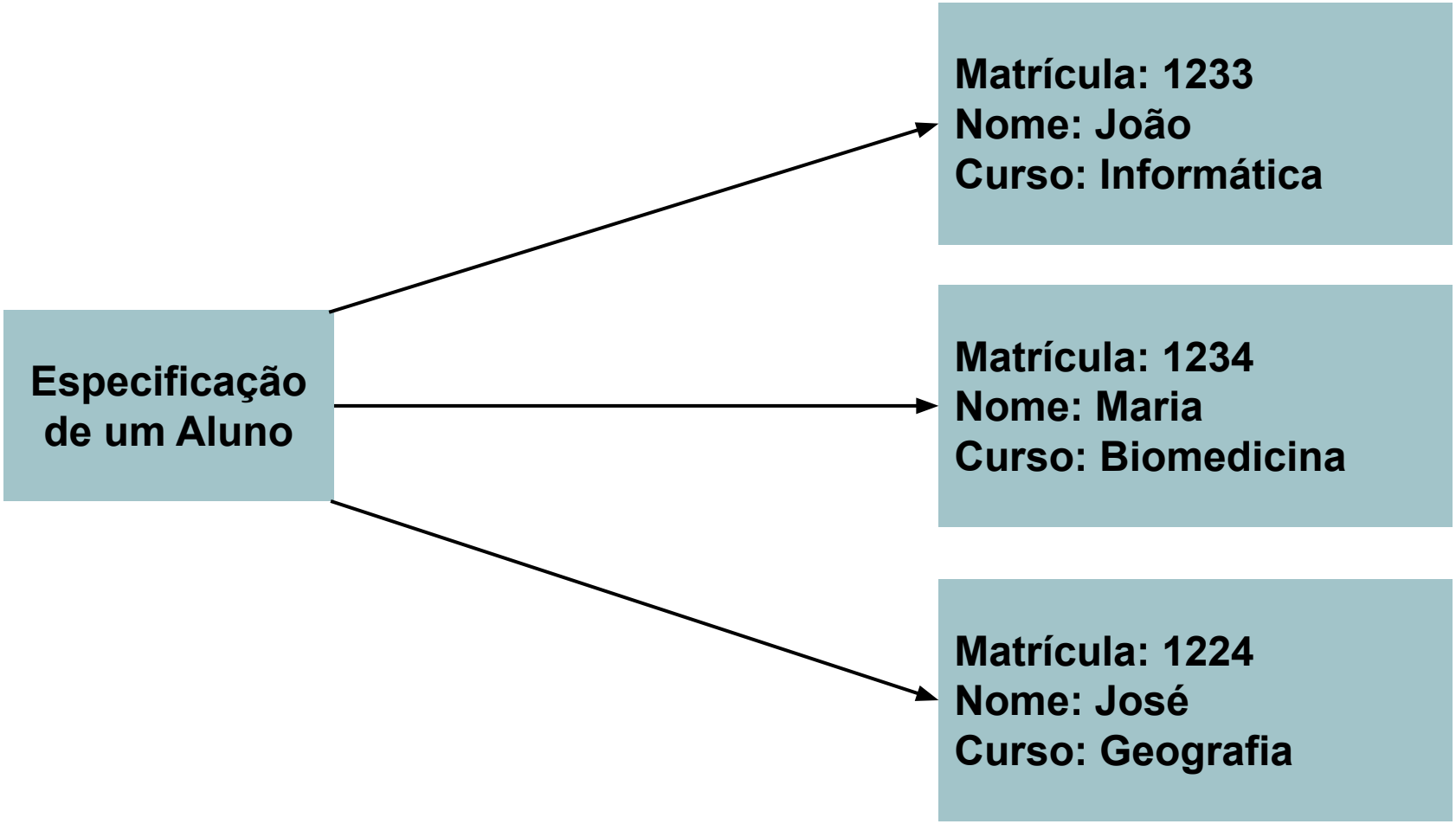
ORIENTAÇÃO A OBJETOS BÁSICA

- O que todo aluno faz que é importante? Isto é, quais ações que é possível realizar?
  - visualizar nota;
  - realizar tarefa;
  - realizar matrícula;
  - solicitar documento;

ORIENTAÇÃO A OBJETOS BÁSICA

- Com isso, temos o projeto de um sistema escolar focado no aluno. Podemos pegar esse projeto e acessar sua nota?
- Não. O que temos ainda é o projeto. Antes, precisamos construir um Aluno para poder acessar o que ele tem e pedir a ele que faça algo.

ORIENTAÇÃO A OBJETOS BÁSICA



ORIENTAÇÃO A OBJETOS BÁSICA

- A palavra **classe** vem da taxonomia da biologia. Todos os seres vivos de uma mesma classe biológica têm uma série de **atributos** e **comportamentos** em comum, mas não são iguais, pois podem variar nos valores desses **atributos** e como realizam esses **comportamentos**.

ORIENTAÇÃO A OBJETOS BÁSICA

- Apesar de o papel do lado esquerdo especificar um Aluno, essa especificação é um Aluno? Nós podemos realizar alguma ação como realizar matrícula? Não. Utilizamos a especificação do Aluno para poder criar instâncias que realmente são alunos, nas quais podemos realizar as operações que criamos.
- Ao projeto do Aluno, isto é, à definição do Aluno, damos o nome de **classe**. Ao que podemos construir a partir desse projeto; os alunos de verdade, damos o nome de **objetos**.

ORIENTAÇÃO A OBJETOS BÁSICA

- Uma classe em Java:

```
public class Aluno{
    String nome;
    String curso;
    String matricula;
}
```

- Por enquanto, declaramos o que todo aluno deve ter. Esses são os atributos que os alunos quando criadas terão.

ORIENTAÇÃO A OBJETOS BÁSICA

- Criando e usando um objeto:
  - Para criar (construir, instanciar) uma Conta , basta usar a palavra-chave **new** .

```
public class Programa {
    public static void main(String[] args) {
        Aluno novoAluno = new Aluno();
    }
}
```

ORIENTAÇÃO A OBJETOS BÁSICA

- Criando e usando um objeto:
  - Por meio da variável novoAluno, podemos acessar o objeto recém criado para alterar seu nome, seu curso, etc.:

```
public class Programa {
    public static void main(String[] args) {
        Aluno novoAluno = new Aluno();
        novoAluno.nome = "Maria";
        novoAluno.curso = "Biomedicina";
        novoAluno.matricula = "1234";
        System.out.println("Nome do aluno: " + novoAluno.nome);
    }
}
```

ORIENTAÇÃO A OBJETOS BÁSICA

- **Métodos:**
  - Dentro da classe, também declararemos o que cada aluno faz e como isso é feito - os comportamentos que cada classe tem.
  - Essas funções são chamadas de **métodos**, pois é a maneira de fazer uma operação com um objeto.

```
public class Aluno{
    String nome;
    String curso;
    String matricula;

    void realizarMatricula(String disciplina){
        System.out.println("Matrícula realizada na disciplina: " + disciplina);
    }
}
```

ORIENTAÇÃO A OBJETOS BÁSICA

- **Métodos com retorno:**

```java
public class Aluno{
    String nome;
    String curso;
    String matricula;

    void realizarMatricula(String disciplina){
        System.out.println("Matrícula realizada na disciplina: " + disciplina);
    }

    boolean enviarTarefa(String tarefa){
        if (tarefa.isEmpty()){
            return false;
        }else{
            return true;
        }
    }
}
```

ORIENTAÇÃO A OBJETOS BÁSICA

- **Realizando testes:**

```java
public class Programa {
    public static void main(String[] args) {
        Aluno novoAluno = new Aluno();
        novoAluno.nome = "Maria";
        novoAluno.curso = "Biomedicina";
        novoAluno.matricula = "1234";
        System.out.println("Nome do aluno: " + novoAluno.nome);
        novoAluno.realizarMatricula("POO");
        boolean status = novoAluno.enviarTarefa("Lista de Atividade 1");
        System.out.println("Status da tarefa: " + status);

    }
}
```

Atividade prática

Atividade prática

1. Crie uma classe chamada Produto com os seguintes atributos: descrição, preço e quantidade;
2. Crie métodos para a classe Produto: verQuantidade, verDescrição, verPreço;
3. Crie um método com retorno para a classe Produto utilizando o atributo preço;
4. Crie 3 objetos do tipo Produto e faça a chamada dos métodos contidos em cada objeto.

Dúvidas, sugestões?